Sansad Combat

A browser fighting game with a framework-free simulation core, a 3D stage, and an AI-assisted build workflow

~10.4k	9	9	3	3
lines of TypeScript	data-driven fighters	bespoke special-move FX	3D arenas	AI difficulty levels

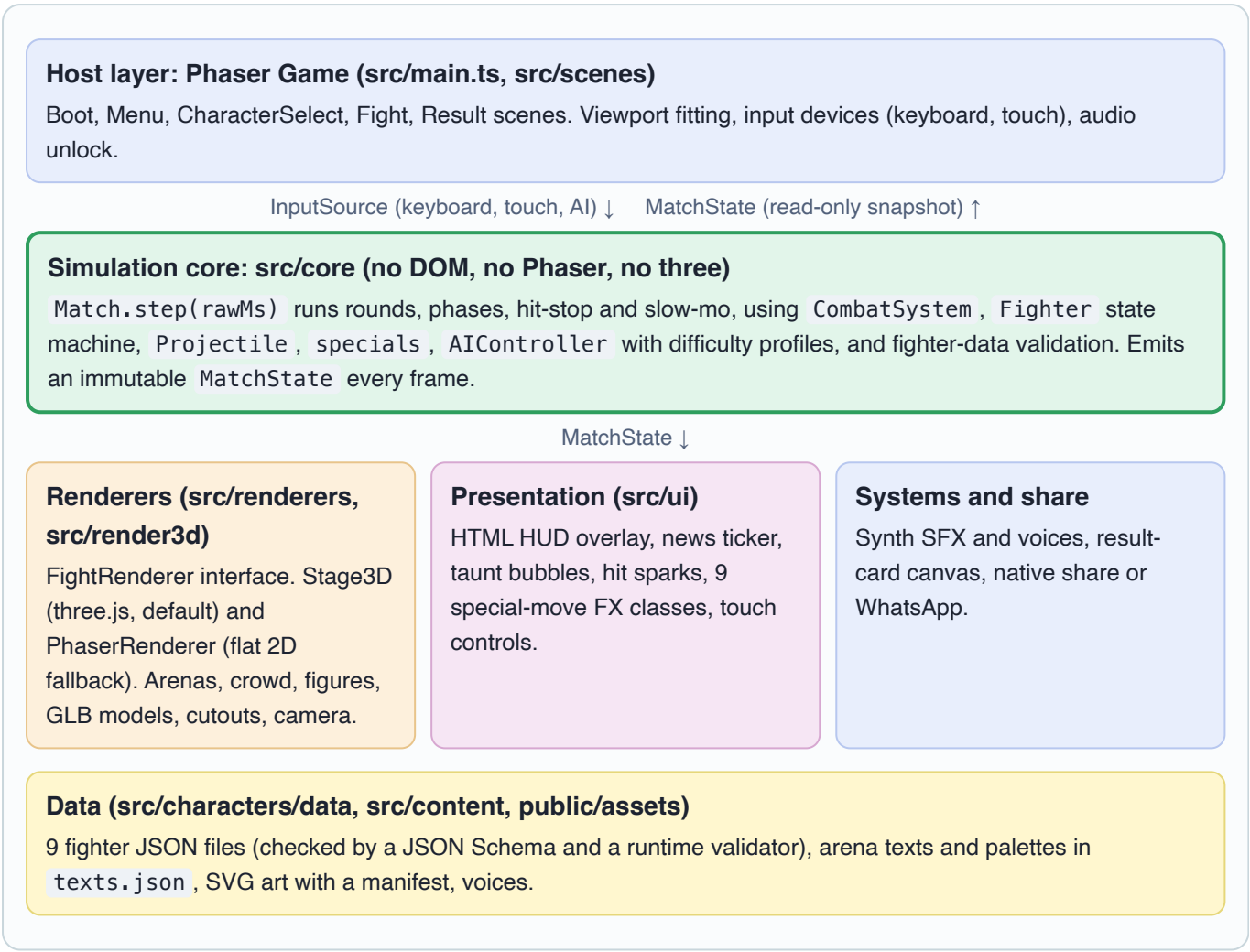
1. What it is

Sansad Combat is a satirical 2D-fighter-in-a-3D-world that runs in any browser, on desktop (keyboard) and on phones (touch controls, installable as a PWA). It has a roster of 9 fighters in two factions (NDA and INDI Alliance), VS CPU with three difficulty levels, local 2-player, three arenas, per-character specials, grabs and slams, voices, synthesised sound effects, and a shareable result card. It is deployed on Vercel.

2. Tech stack

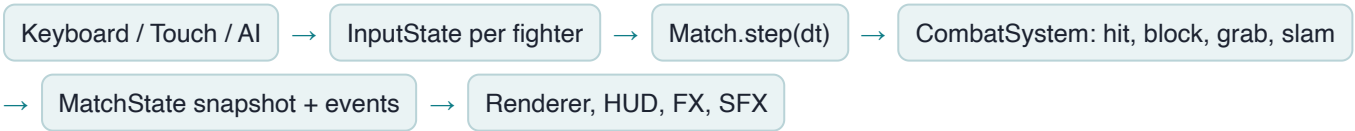
Concern	Choice	Why
Language	TypeScript (strict), ES modules	Types make the data-driven design safe to refactor
Build	Vite 6, manual chunks for Phaser and three	Fast dev loop, separate cacheable vendor bundles
Scenes, input, 2D	Phaser 3	Scene lifecycle, multi-touch input, scaling, 2D fallback renderer
3D stage	three.js	Arenas, crowd, lights, fog, camera work for specials, 3D fighters
Audio	Web Audio API (synthesised SFX) plus short voice clips	No heavy audio assets; unlocks on first user gesture for mobile
Characters	JSON plus JSON Schema, SVG art, optional GLB	New fighter is data, not code
Deploy	Vercel, immutable caching for assets, security headers	Zero-config static hosting

3. Architecture at a glance



Key rule: dependencies point one way. Everything outside `src/core` only reads `MatchState`. The compiler enforces it: `npm run build` runs `tsc -p src/core` with no DOM or engine types available, so a violation fails the build.

4. How a frame works



- **Frame clamp and hit-stop.** `MAX_FRAME_MS` bounds big frame gaps (tab switches); hit-stop and KO slow-motion are applied inside the core so every consumer stays in sync.
- **Controllers are pluggable.** A fighter is driven by an `InputSource` (keyboard, touch, or the `AIController`); the same interface leaves room for a network source.
- **AI.** A small state machine (approach, attack, defend, retreat) with perception of distance, reach and incoming threats, tuned by easy, normal and hard profiles (reaction time, aggression, block and special chances).
- **Events, not polling.** Hits, blocks, KOs and special start/end are emitted as events; SFX, sparks and camera shake subscribe to them.

5. Data-driven fighters

Each fighter is one JSON file: stats, body box, pose set, move frame data (startup, active, recovery, damage, hit-stun, block-stun, knockback, hitbox), special definition, taunts, win quote, faction, a 3D `figure` description, and an SVG rig description. `FighterRegistry` loads every file in the folder via a Vite glob and validates it against the schema; invalid data logs a warning instead of crashing. Balance and content changes need no engine code.

Three visual tiers per fighter

Tier	Used when
GLB model	The JSON has a `model` block; animation clips are crossfaded by action, root motion stripped, capsule shown until the file loads
Procedural figure	The JSON has `figure`; a 3D body is built in code and posed from the rig frame
SVG cutout	Fallback; hand-authored SVG art sliced on a standard skeleton and animated by an angle-based rig

6. Rendering

- **Layered canvases.** A transparent Phaser canvas sits over a three.js canvas. Phaser draws UI and 2D FX; three draws the world. `stage.project()` maps world points to screen so 2D overlays line up with 3D bodies (1 world unit = 100 game px).
- **Renderer abstraction.** `FightRenderer` is implemented by `Stage3D` and `PhaserRenderer`. The three renderer is the default; without WebGL the game falls back to Phaser automatically, and `?renderer=phaser` forces it.
- **Arenas.** Each arena (Sansad, India Gate, Jantar Mantar) is a builder returning an `Arena` object (scene root, background, fog, update, cheer, placards). They are built lazily and toggled by visibility. Placard and ticker text lives in `texts.json`.
- **Cinematics.** Specials trigger a cutscene camera and slow-motion through `windingUp(state)` from the core.

7. Special-move FX framework

Specials are projectile definitions with an `fx` id. A factory maps the id to a class (`ConstitutionFx`, `BulldozerFx`, `CycleFx`, `LathiFx`, `PanjaFx`, `BroomStormFx`, `MicDropFx`, `BaarishFx`, `JaaduFx`) built on shared helpers in `kit.ts` (puffs, gathers, floating text, `finalState`). Effects read state only and track projectiles by id, since a projectile is removed the frame it dies.

8. Mobile, PWA and deployment

- Landscape fullscreen web-app manifest, touch controls with multi-touch, fullscreen, pause and back buttons.
- iPhone Safari bar hiding handled with a visual-viewport fit and a scroll trick; audio unlocked on the first gesture.
- Vercel: `npm ci` then `npm run build`; `/assets/*` cached for a year as immutable (changed art is renamed, not overwritten), manifest always revalidated, `nosniff` and referrer-policy headers.

9. How it was built: an AI-assisted workflow

The game was built in stages by directing an AI coding agent (GitHub Copilot agent mode in VS Code), with me owning the product decisions, architecture rules, review and testing. The repository records this: staged commits, written architecture rules and reusable agent and skill files.

Timeline (git history, 27 to 29 Oct)

Phase	What was built
Baseline commit	Snapshot of the working game: Phaser scenes, framework-free `src/core` (extracted by refactor and checked against the old code on 120 seeded AI-vs-AI matches), three.js stage, hall arena, crowd, HTML HUD, procedural 3D figures, optional GLB path, grab and slam, 3 arenas
Stage 1	Special-move FX framework, knockdown, multi-hit projectiles, Rahul's Constitution Shockwave
Stages 2 to 6	One cinematic special per character: Bulldozer Rush, Cycle Cyclone, Lathi Charge, Panja Power, Jhaadu Storm, Mic Drop, Chai Ki Baarish
Cleanup	Removed unused views and their code; renamed factions and characters; art ids made stable
Polish and ship	Rename to Sansad Combat, mobile chrome, synthesised SFX, voices, Didi (Jaadu special), iPhone Safari bar hiding, Vercel deploy

The agent setup in the repo

File	Role
`AGENTS.md`	Always-on project brief: commands, architecture rules, conventions
`.github/instructions/core-purity`	Applies automatically when editing `src/core/**`: no DOM or engine imports
`agents/architect-reviewer`	Read-only reviewer for layer boundaries, per-frame cost, fallback renderer health
`agents/fighter-designer`	Edits fighter JSON and checks balance against existing roster
`agents/qa-playtester`	Runs build, headless simulations and Playwright smoke tests
`skills/add-fighter`	Step-by-step: JSON, art, manifest, voice, verify
`skills/add-special-fx`	FX class, registration, projectile tracking rules
`skills/add-arena`	Arena builder, registry, texts, verify

The loop I followed for every feature

Spec the behaviour → Agent implements in a small stage → Typecheck + core boundary build → Headless sim / Playwright run → Screenshot review → Commit and tag

Git tags (`before-cleanup`, `after-cleanup`, `after-rename`) mark safe rollback points before large changes.

10. Design decisions and trade-offs

Decision	Reasoning
Simulation separated from rendering	Lets the AI and balance run headless, lets the renderer be swapped (3D to 2D fallback) and makes refactors checkable by comparing traces
Everything about a fighter in JSON	Adding or rebalancing a character is a data change; one schema validates all
Two canvases instead of one engine	Phaser is strong at scenes and input, three at 3D; the cost is projecting coordinates between them
Synthesised SFX	Small bundle and no licensing issues; only a block clip and voice lines are audio files
Immutable asset caching	Fast repeat loads; the cost is renaming files when art changes
Fallback renderer	Devices without WebGL still play; keeping it alive requires the interface discipline

11. Honest limitations

- No automated unit-test suite in the repo; verification is typecheck, build, headless simulations and Playwright smoke scripts.
- The AI uses `Math.random` for reaction jitter, so AI matches are not bit-for-bit reproducible without seeding it.
- Online multiplayer is not implemented; the `InputSource` abstraction is the intended hook.
- Character names and faction names are satirical parody; this is an entertainment project.

Source: github.com/AkashSrivastava1993/sansad-combat | Live: sansad-combat.vercel.app